C++: Templates

Carsten Gips (HSBI)

TL;DR
In C++ können Funktionen über **Funktions-Templates** definiert werden. Dafür stellt man ein `template <typename T>` mit einer Aufzählung `typename T` aller Template-Parameter der Funktionsdefinition voran. In der Funktion kann man dann den Typ `T` wie einen normalen anderen Typ nutzen.
Funktions-Templates sind (vollständig) spezialisierbar. Dazu wiederholt man die komplette Funktionsdefinition (inkl. der dann leeren Template-Deklaration `template <>`) und legt alle Template-Parameter über die Funktionssignatur fest. Alle Spezialisierungen müssen nach dem eigentlichen ("primären") Funktions-Template formuliert werden.
Funktions-Templates sind überladbar mit "normalen" Funktionen und anderen Funktions-Templates.
Beim Aufruf kann man die Template-Parameter entweder selbst festlegen (über eine Auflistung der Typen in spitzen Klammern hinter dem Funktionsnamen) oder den Compiler inferieren lassen. Dabei wird die am besten "passende" Variante genutzt:

1. Zuerst die **exakt** passende normale Funktion,
2. dann ein passendes spezialisiertes Template (bei mehreren passenden spezialisierten Templates das am meisten spezialisierte Template),
3. dann das allgemeine ("primäre") Template,
4. ansonsten die normale Funktion mit impliziten Casts.

In C++ können Klassen als **Klassen-Templates** definiert werden. Dafür stellt man ein `template <typename T>` mit einer Aufzählung `typename T` aller Template-Parameter der Klassendefinition voran. In der Klasse kann man dann den Typ `T` wie einen normalen anderen Typ nutzen. Bei der Implementierung der Methoden außerhalb der Klassendeklaration muss die Template-Deklaration (`template <typename T>`) wiederholt werden.
Klassen-Templates sind spezialisierbar (vollständig und partiell). Dazu wiederholt man die komplette Klassendefinition (inkl. der Template-Deklaration `template <typename T>`) und entfernt aus der Template-Deklaration alle Typen, die man konkret festlegen möchte. Hinter dem Klassennamen werden dann in spitzen Klammern alle Typen (verbleibende Typ-Parameter aus der Template-Deklaration sowie die konkretisierten Typen) in der Reihenfolge angegeben, wie sie im primären Template vorkamen. Spezialisierungen müssen nach dem eigentlichen ("primären") Klassen-Template formuliert werden.
Klassen- und Funktions-Templates können gemischt werden.
Bei der Instantiierung werden die Template-Parameter in spitzen Klammern hinter dem Klassennamen spezifiziert. Template-Parameter können einen konkreten (aufzählbaren) Typ haben (beispielsweise `int`). Template-Parameter können Default-Werte haben.
Im Unterschied zu Java gibt es **keine** Type-Erasure. Der C++-Compiler stellt je **instantiiertem** Template eine *konkrete* Funktion bzw. Klasse bereit! Im resultierenden Code sind also nur diejenigen Funktionen und Klassen enthalten, die aus einem tatsächlichen Aufruf resultieren, das Template selbst ist nicht im Code enthalten. Dies gilt auch für Bibliotheken, weshalb sich diese beiden Konzepte etwas "quer liegen".
Videos

- VL C++: Vererbung und Polymorphie

---

1 Vergleichsfunktion für zwei Integer?

```
bool cmp(const int &a, const int &b) {
    return a<b;
```

```
}
```

- Und für **double**?
- Und für string?
- ...

=> Präprozessor-Makro?

=> Funktionen überladen?

- **Überladen von Funktionen**:
  - Ähnliche Funktionalität für unterschiedliche Datentypen
  - Mühselig, wenn exakt gleiche Funktionalität!
- (bessere) **Antwort**: Funktions-Templates
  - Templates: Funktionen mit parametrisierten Datentypen
  - Deklaration/Definition für (zunächst) unbestimmte Datentypen
  - Bei Verwendung der Funktion:
  - Konkretisierung der Datentypen
  - Compiler erzeugt automatisch passende Funktionsinstanz

2 Definition von Funktions-Templates

```
template <typename T>
bool cmp(const T &a, const T &b) {
    return a<b;
}
```

- Statt typename kann auch **class** geschrieben werden
- Konvention:
  - typename wenn sowohl Klassen als auch Basistypen
  - **class** falls eher Klassen-Typen

  => **class** gilt als veraltet, deshalb immer typename verwenden!
- Bei mehreren Typen "typename NAME" wiederholen
  (Komma-separierte Liste in < und >)

  beispielsweise so: (Achtung, soll nur die Verwendung demonstrieren, hat sonst keinen Sinn)

  ```
  template <typename T1, typename T2, typename T3>
  T1 cmp(const T2 &a, const T3 &b) {
      return a<b;
  }
  ```

Vorsicht: Im Beispiel oben muss operator< für die verwendeten Typen T implementiert sein!
(sonst Fehler zur Compile-Zeit)

3 Bestimmung der Template-Parameter I: Typ-Inferenz

Das Funktions-Template wird wie eine normale Funktion aufgerufen … => Der Compiler inferiert Typen und erzeugt passende Funktionsinstanz(en).

```
template <typename T>
bool cmp(const T &a, const T &b) {
    return a<b;
}

int main() {
    cmp(3, 10);                          // cmp(int, int)
    cmp(2.2, 10.1);                      // cmp(double, double)
    cmp(string("abc"), string("ABC"));   // cmp(string, string)
    cmp(3, 3.4);                         // Compiler-FEHLER!!!
}
```

Vorsicht bei Typ-Inferenz: Typen müssen *exakt* passen!

4 Bestimmung der Template-Parameter II: Explizite Angabe

```
template <typename T>
bool cmp(const T &a, const T &b) {
    return a<b;
}

int main() {
    cmp<int>('a', 'A');      // cmp(int, int)
    cmp<int>(3, 3.4);        // cmp(int, int)
}
```

Bei expliziter Angabe der Typen beim Aufruf (`cmp<int>`) kann der Compiler automatisch casten.

5 Typ-Inferenz und explizite Bestimmung mischen

- Compiler nutzt die vorgegebenen Typ-Parameter, …
- … inferiert die restlichen, und …
- … castet notfalls die Parameter

```
template <typename T1, typename T2, typename T3>
void fkt(T2 a, T3 b, T2 c, int d) { ... }


int main() {
    fkt<void*, int>(42, "HUHU", 'a', 99);
}
```

=> In Parameterliste nicht vorkommende Typ-Parameter **explizit** angeben!

- Reihenfolge der Angabe der Typen in spitzen Klammern beim Aufruf wie in Template-Deklaration
- Wenn ein Typ-Parameter nicht in der Parameterliste der Funktion vorkommt, ist eine Inferenz für den Compiler unmöglich. Deshalb **muss** dieser Typ beim Aufruf explizit in der Liste mit den spitzen Klammern angegeben werden!
- Im Beispiel oben:
  - `fkt<a, b, c>(...)`: `a` wäre der Typ für `T1`, `b` für `T2`, `c` für `T3`
  - Mit `fkt<..., int>(...)` beim Aufruf wird `T2` zu `int` und damit für Parameter `c` der Char als `int` interpretiert (`T3` wird inferiert)

    Ohne `<..., int>` beim Aufruf gibt es ein Problem beim Erkennen von `T2`: `int` vs. `char` (`a=42`, `c='a'`)...

6 Typ-Inferenz funktioniert nicht immer!

```
template <typename T>
T zero() {
    return 0;
}

int main() {
    int x, y;
    x = zero(); // Fehler: couldn't deduce template parameter 'T'
    y = zero<int>(); // korrekter Aufruf
}
```

Die Funktion hat keine Parameter - der Compiler hat also keine Chance, den Typ `T` zu inferieren. In diesem Fall *muss* der Typ beim Aufruf explizit angegeben werden.

7 Spezialisierung von Funktions-Templates

```
// Primaeres Template
template <typename T>
bool cmp(const T &a, const T &b) {
    return a<b;
}
```

```
// Spezialisiertes Template
template <>
bool cmp<int>(const int &a, const int &b) {
    return abs(a)<abs(b);
}
```

Spezialisierte Templates **nach** "primärem" Template definieren

Achtung: Reihenfolge der Deklaration/Definition ist *wichtig*. Immer zuerst das allgemeine ("primäre") Template definieren, danach dann die Spezialisierungen! Anderenfalls gibt es "seltsame" Fehlermeldungen vom Compiler oder sogar seltsames Verhalten.

Achtung: Im Unterschied zu Klassen-Templates können Funktions-Templates nur **vollständig** spezialisiert werden (d.h. bei mehreren Template-Parametern müssen dann *alle* Template-Parameter konkret spezifiziert werden)!

Anmerkung: Die Angabe der Typen in spitzen Klammern nach dem Funktionsnamen ist freiwillig, so lange *alle* Typ-Parameter in der Parameterliste der Funktion auftauchen. Man könnte die obige Spezialisierung also auch so schreiben (`cmp(` statt `cmp<int>(`):

```
// Spezialisiertes Template
template <>
bool cmp(const int &a, const int &b) {
    return abs(a)<abs(b);
}
```

7.1 Alternativ: Überladen der Funktions-Templates mit normalen Funktionen

Überladen mit normalen Funktionen funktioniert wie bei spezialisierten Templates, d.h. auch hier zuerst das primäre Template definieren, danach eventuelle Spezialisierungen und danach Überladungen mit normalen Funktionen.

Allerdings gibt es Unterschiede für eventuell nötige Typumwandlungen der Parameter beim Aufruf der Funktionen:

- In gewöhnlichen Funktionen sind automatische Typumwandlungen möglich
- In (spezialisierten) Templates sind keine automatischen Typumwandlungen erlaubt (sofern man mit Typ-Inferenz arbeitet, d.h. die Template-Typen nicht beim Aufruf explizit angegeben werden)

```
template <typename T>
bool cmp(T a, T b) {
    return a<b;
}

bool cmp(int a, int b) {
    return abs(a)<abs(b);
}

int main() {
    cmp(3, 6);          // true:  überladene normale Funktion
    cmp(3, 3.4);        // FALSE: überladene normale Funktion (Cast)
    cmp<int>(3, 3.4);   // FALSE: Template
}
```

7.2 Aufruf: Compiler nimmt die am besten "passende" Variante:

- Keine Template-Parameter beim Aufruf angegeben (d.h. Typ-Inferenz):
    1. Zuerst **exakt** passende normale Funktion,
    2. dann passendes spezialisiertes Template (bei mehreren passenden spezialisierten Templates das am meisten spezialisierte Template, ohne Casts),

3. dann das allgemeine ("primäre") Template (ohne Casts),
4. ansonsten normale Funktion mit impliziten Casts

- Template-Parameter beim Aufruf angegeben: am besten passendes Template

Hinweis: Durch reine Deklaration von Spezialisierungen (d.h. ohne die entsprechende Implementierung) lässt sich die Instantiierung einer Templatefunktion für bestimmte Typen verhindern. Beispiel:

```
template <typename T>
bool cmp(const T &a, const T &b) {
    return a<b;
}

template <>
bool cmp<int>(const int &a, const int &b);
```

Damit könnte man die `cmp`-Funktion nicht mehr für `int` benutzen (Compiler- bzw. Linker-Fehler).

Konsole: funktionsTemplates.cpp

8 Klassen-Templates in C++

```
template <typename T>
class Matrix {
    Matrix(unsigned rows = 1, unsigned cols = 1);
    vector<vector<T> > xyField;
};
```

Hinweis: Template-Parameter innerhalb von Template-Parametern verursachen bei den schließenden spitzen Klammern u.U. Parser-Probleme. Diese lassen sich durch ein extra Leerzeichen (hat sonst keine Funktion!) umgehen: Statt `vector<vector<T>> xyField;` besser `vector<vector<T> > xyField;` schreiben.

```
int main() {
    Matrix<int> m1;
    Matrix<double> m2(12, 3);
}
```

Template-Parameter gehören zur Schnittstelle und müssen bei der Instantiierung angegeben werden. `Matrix m;` würde im obigen Beispiel **nicht** funktionieren.

```
template <typename T>
Matrix<T>::Matrix(unsigned rows, unsigned cols) { ... }
```

Beispiel: matrix.cpp

8.1 Klassen-Templates in C++ (Variante mit Konstanten)

```
template <typename T, unsigned rows, unsigned cols>
class Matrix {
    Matrix();
    vector<vector<T> > xyField;
};
```

Template-Parameter können neben Typen auch konstante Werte (Basisdatentypen, außer **`float`** und **`double`**) sein. Innerhalb der Klasse `Matrix` kann auf die Werte von `rows` und `cols` zugegriffen werden.

Achtung: `rows` und `cols` sind **keine** Attribute der Klasse `Matrix`!

Hinweis: Konstanten als Template-Parameter funktioniert auch bei Funktions-Templates.

```
int main() {
    Matrix<int, 1, 1> m1;
    Matrix<double, 12, 3> m2;
    Matrix<string, 1, 1> m3;
}
```

Beispiel: matrix2.cpp

8.2 Beispiel: Konstanten als Template-Parameter

```
template <int I>
void print() {
    cout << I;
}

print<5>();
```

```
template <unsigned u>
struct MyClass {
    enum { X = u };
};

cout << MyClass<2>::X << endl;
```

- Konstante muss explizit übergeben werden
- Wert muss eine zur Compile-Zeit bekannte Konstante sein
- Nur aufzählbare Typen für derartige Konstanten erlaubt
  (z.B. **`int`**, aber nicht **`double`**)

Anmerkung: Durch Konstruktion mit dem anonymen `enum` in der Klasse `MyClass` wird der Wert der Konstanten "gespeichert" und kann später (von außen) abgefragt werden. (Innerhalb der Klasse selbst können Sie natürlich jederzeit auf den Wert von `u` zugreifen.)

Wollte man dies über ein normales Attribut erledigen, sähe der Code deutlich komplexer aus (zusätzlich zur oben gezeigten Variante mit dem `enum` einmal als statisches Attribut `Y` und einmal als "normales" Attribut `Z`):

```
template <unsigned u>
struct MyClass {
    enum { X = u };
```

```
    static unsigned Y;
    unsigned Z;

    MyClass() : Z(u) {}
};

template <unsigned u>
int MyClass<u>::Y = u;

int main() {
    cout << MyClass<2>::X << endl;
    cout << MyClass<2>::Y << endl;
    cout << MyClass<2>().Z << endl;
}
```

Falls man mit `::` zugreifen wollte, müsste das Attribut `static` sein und entsprechend außerhalb der Klasse initialisiert werden. Für ein "normales" Attribut braucht man dann einen extra Konstruktor und muss den Aufruf dann extra klammern: `MyClass<2>().Z` statt `MyClass<2>.Z`.

Die Variante mit dem `enum` werden Sie entsprechend sehr häufig in C++ finden!

8.3 Klassen-Templates mit Defaults

```
template <typename T = int, unsigned rows = 1, unsigned cols = 1>
class Matrix {
    Matrix();
    vector<vector<T> > xyField;
};
```

```
int main() {
    Matrix<> m1;    // Leere spitze Klammern Pflicht!
    Matrix<double, 12, 3> m2;
    Matrix<string> m3;
}
```

Leere spitze Klammern bei Klassen-Templates mit Default-Parameter Pflicht!

Hinweis: Defaults für Template-Parameter waren zunächst nur für Klassen-Templates erlaubt. Seit C++11 kann man solche Defaults auch bei Funktions-Templates einsetzen.

Beispiel: matrix3.cpp

9 Klassen-Templates in C++ spezialisieren

```
template <typename T>
class Matrix {
    Matrix(unsigned rows, unsigned cols);
    vector< vector<T> > xyField;
};
```

```
template <>
class Matrix<uint> {
    Matrix(unsigned rows, unsigned cols);
    vector< vector<uint> > xyField;
};
```

Hinweis auf Implementierung außerhalb der Klasse

ACHTUNG: Implementierung außerhalb der Klasse: Bei den Methoden des voll spezialisierten Templates das "`template<>`" **weglassen**! Alles andere ist ein Syntax-Fehler.

Der Grund dafür bleibt ein Geheimnis des C++-Standards … ;-)

```
// Implementierung fuer primaeres Template
template <typename T>
Matrix<T>::Matrix(unsigned rows, unsigned cols) { ... }

// Implementierung fuer vollstaendig spezialisiertes Template
Matrix<uint>::Matrix(unsigned rows, unsigned cols) { ... }
```

9.1 Partielle Spezialisierung

```
template <typename T1, typename T2>
class Array {
    Array();
    vector<T1> v;
    vector<T2> w;
};
```

```
template <typename T>
class Array<T, int> {
    Array();
    vector<T> v;
    vector<int> w;
};
```

ACHTUNG: Implementierung außerhalb der Klasse: Bei den Methoden des partiell spezialisierten Templates muss das "`template<T>`" wieder benutzt werden!

```
// Implementierung fuer primaeres Template
template <typename T1, typename T2>
Array<T1, T2>::Array() {}

// Implementierung fuer partiell spezialisiertes Template
template <typename T>
Array<T, int>::Array() {}
```

9.2 Vergleich verschiedene Spezialisierungen

Allgemeine Templates vs. partiell spezialisierte Templates vs. vollständig spezialisierte Templates

```
// Primaeres (allgemeines) Template
template <unsigned line, unsigned column>
class Matrix {
public:
    Matrix();
};

// Partiell spezialisiertes Template
template <unsigned line>
class Matrix<line, 1> {
public:
    Matrix();
};

// Vollstaendig spezialisiertes Template
template <>
class Matrix<3, 3> {
public:
    Matrix();
};


// Aufrufe
int main() {
    Matrix<3, 4> matrix;        // allg. Template
    Matrix<20, 1> vector;       // partiell spez. Templ.
    Matrix<3, 3> dreiKreuzDrei; // vollst. spez. Templ.
}


// Implementierung
template <unsigned line, unsigned column>
Matrix<line, column>::Matrix() { cout << "allgemeines Template" << endl; }

template <unsigned line>
Matrix<line, 1>::Matrix() { cout << "partiell spezialisiertes Template" << endl
    ; }

Matrix<3, 3>::Matrix() { cout << "vollstaendig spezialisiertes Template" <<
    endl; }
```

Regel: Das am meisten spezialisierte Template wird verwendet.

10 Mischen von Klassen- und Funktions-Templates

Sie können innerhalb eines Klassen-Templates auch ein Funktions-Template (Methoden-Template) definieren. Bei der Implementierung außerhalb der Klasse müssen entsprechend alle Template-Deklarationen wiederholt werden!

```
template <typename T, unsigned n>
class Array {
public:
    enum { size = n };
```

```
5       template <typename C >
6       void copy_from(const C &c);
7   private:
8       T data[n];
9   };
10
11
12  template <typename T, unsigned n>
13  template <typename C>
14  void Array<T,n>::copy_from(const C &c) { ... }
```

11 Templates: Java vs. C++

- Templates sind nur **Schablonen**!

  Die Definitionen der Templates werden nicht in den Object-Code kompiliert! Erst bei der Instantiierung von Templates wird durch den Compiler eine passende Instanz erzeugt und im Object-Code zur Nutzung abgelegt.

- **Unterschied zu Java**
  - C++: Für **jeden** Aufruf/Typ eine passende **Instanz** (!)
  - Java: Nur **eine** Klasse mit gemeinsamen Obertyp

- Offener Code: **Templates im `.h`-File implementieren!**

  Ohne die Template-Definition kann der Compiler keine Instanzen anlegen!

- Bibliotheken und Templates passen nicht recht

  Templates werden immer bei der ersten Verwendung instantiiert! Wird ein Template nicht im zu kompilierenden Code verwendet, dann generiert der Compiler auch nichts, was man in den Objektdateien finden könnte …

  - Nur instantiierte Templates sind in einer dynamischen/statischen Bibliothek enthalten!
  - Falls Einsatz nur für wenige Typen vorgesehen => Explizite Instantiierung:
    * Entweder mit "`template`": `template class Matrix<5,5>;`, oder
    * mit "`typedef`": `typedef Matrix<5,5> Matrix5x5;`

      => Dann aber nur in exakt diesen Versionen in der Bibliothek enthalten und entsprechend nur so nutzbar (sofern die Template-Definition nicht zur Verfügung steht)

12 Wrap-Up

- Generische Programmierung (Funktions-Templates)
  - `template <typename T>` der Funktionsdefinition voranstellen

  - Funktions-Templates sind spezialisierbar und überladbar
  - Aufruf: Compiler nimmt die am besten “passende” Variante

- Generische Programmierung (Klassen-Templates)
  - Funktionsweise analog zu Funktions-Templates
  - Bei Implementierung außerhalb der Deklaration: Template-Deklaration mitführen!
  - Klassen-Templates lassen sich partiell spezialisieren

- Compiler stellt je instantiiertes Template eine konkrete Funktion/Klasse bereit

13 Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Unterschied zu Java bei der Nutzung von Templates
- k3: Erstellen und spezialisieren von Funktions-Templates
- k3: Unterschied zwischen überladenen Funktionen und Funktions-Templates
- k3: Aufruf (Nutzung) von Funktions-Templates
- k2: Unterschied zu Java bei der Nutzung von Templates
- k3: Erstellen und spezialisieren von Klassen-Templates
- k3: Nutzung von Methoden-Templates innerhalb von Klassen-Templates
- k3: Aufruf (Nutzung) von Klassen-Templates

Challenges

Funktions-Templates

- Wie kann eine Funktion als Funktions-Template definiert werden?
- Wie wird ein Funktions-Template benutzt, d.h. wie wird es aufgerufen? Worin liegt der Unterschied zu Java?
- Wie kann ein Funktions-Template spezialisiert werden? Was ist dabei zu beachten?
- Kann man Funktions-Templates überladen?
- Worin liegt der Unterschied zwischen einem spezialisierten Funktions-Template und einem überladenen Funktions-Template?

Funktions-Templates in C++

1. Schreiben Sie in C++ eine Funktion `invert()`, die zu einem übergebenen numerischen Wert den negativen Wert zurückgibt. Realisieren Sie die Funktion als Funktions-Template.

   Testen Sie Ihre Implementierung mit den Typen **`int`**, **`double`** und **`short`**.

2. Spezialisieren Sie das eben erstellte Funktions-Template, so daß `invert()` auch für `string` aufgerufen werden kann. In diesem Fall soll der String umgekehrt werden und zurückgeliefert werden, d.h. für die Eingabe von "abcde" soll "edcba" zurück geliefert werden.

   Testen Sie die Funktionen wiederum mit **`int`**, **`double`**, **`short`** und nun auch `string`.

3. Schreiben Sie ein weiteres Funktions-Template `string getType(T t)` mit Template-Spezialisierungen, die den Typ des Parameters `t` als String zurückgibt. Für nicht bekannte Typen soll der String "unbekannter Typ" zurückgeliefert werden.

   Implementieren Sie mind. 3 Spezialisierungen.

   Hinweis: Verwenden Sie an dieser Stelle keine explizite Typüberprüfung (in der Funktion)! Realisieren Sie die Reaktion auf unterschiedliche Parametertypen ausschließlich mit Hilfe von Templates.

4. Erklären Sie folgenden Code-Schnipsel:

```
    // Definition
    template <typename T2, typename T1>
    void f(T1 a, T2 b, int c) {
        ...
    }

    // Aufruf
    f<char *>(99, "Hello World", 42);
```

5. Erklären Sie nun folgenden Code-Schnipsel:

```
template<typename T2, typename T1, typename T3>
void fkt(T2 a, T3 b, T2 c, int d) { ... }
void fkt(int a, int b, int c, int d) { ... }

int main() {
    fkt<int, void*>(42, "HUHU", 'a', 99);
    fkt<int, int, int>(1,2,3,4);
    fkt(1,2,3,4);
}
```

Klassen-Templates und partielle Spezialisierung

Definieren Sie ein Klassen-Template zum Umgang mit Vektoren. Diese sollen Elemente eines unbestimmten Typs (Typ-Variable) aufnehmen. Die Größe des Vektors soll ebenfalls als Typ-Variable in die Template-Definition eingehen.
Definieren Sie den `operator*` für das Skalarprodukt von zwei Vektoren.
Erstellen Sie eine partielle Spezialisierung des Klassen-Templates zur Repräsentation von einstelligen Vektoren (Skalare).
Schreiben Sie ein `main()`-Funktion, instantiieren einige Vektoren und berechnen Sie die Skalarprodukte.

Beispiel aus dem echten Leben

Erklären Sie das folgende Beispiel eines Klassen-Templates `RingBuffer`:

```
template <typename T, size_t size, typename alloc_t = std::allocator<T>>
class RingBuffer {
public:
```

Quellen

Allain, A., und A. Hoffer. o. J. „C Programming and C++ Programming“. Zugegriffen 16. September 2024. https://www.cprogramming.com/.

Breymann, U. 2011. *Der C++ Programmierer*. 2. Auflage. Hanser.

„C and C++ Reference“. o. J. cppreference.com. Zugegriffen 16. September 2024. https://en.cppreference.com/w/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** a10718a (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-25)